Software Requirements Engineering

Chapter 1: The Essential Software Requirement

Software Requirements: What, Why & Who

Table of Contents

1. Current Problems with Requirements
2. What is a Requirement?
3. Levels of Requirements
    - 3.1 Business Requirements
    - 3.2 User Requirements
    - 3.3 Functional Requirements
    - 3.4 Working with the Three Levels
4. Other Types of Requirements
    - 4.1 System Requirements
    - 4.2 Non-Functional Requirements
    - 4.3 Business Rules
    - 4.4 Feature
5. Product vs. Project Requirements
    - 5.1 Product Requirements
    - 5.2 Project Requirements
6. Requirements Development Process
    - 6.1 Elicitation
    - 6.2 Analysis
    - 6.3 Specification
    - 6.4 Validation
7. Requirements Management
8. Importance of Requirements
    - 8.1 Role of Requirements
    - 8.2 Reasons Behind Bad Requirements
    - 8.3 Benefits of a High-Quality Requirements Process
9. References

---

1. Current Problems with Requirements

When requirements aren't handled well, projects run into recurring issues:

- The project's business objectives, vision, and scope were never clearly defined

- Lack of communication — customers were too busy to work with analysts/developers on requirements
- Customers claimed all requirements were critical, so nothing got prioritized
- Developers hit ambiguities and missing information while coding, so they had to guess
- Customers never approved the requirements
- Customers approved requirements for a release, then changed them continually
- Project scope grew as changes were accepted, but the schedule slipped since no extra resources were added and no functionality was removed
- Requested changes got lost — no one tracked the status of a change request
- Developers built functionality that customers requested but no one ever used
- At the end of the project, the specification was satisfied but the business objectives were not

---

2. What is a Requirement?

- A **requirement** is a property a product must have to provide value to a stakeholder
- Requirements specify *what* should be implemented — they describe a system's behavior, properties, or constraints on the development process (not *how* to build it)
- Requirements encompass two views:
  - The **user's view** of external system behavior
  - The **developer's view** of internal characteristics
- Requirements include a **time dimension**:
  - **Present tense** — describing current system capabilities
  - **Near-term** (high priority) or **hypothetical** (low priority) future needs
  - **Past tense** — needs that were once specified, then discarded

---

3. Levels of Requirements

Software requirements exist at **three distinct levels** — Business, User, and Functional. Keeping all three aligned is essential for project success.

3.1 Business Requirements

- Describe **why** the organization is building the system — the business benefits it hopes to achieve
- Focus on the objectives of the organization or requesting customer (requirements gathered from multiple sources can conflict)
- **Example:** An airline wants to cut airport counter staff costs by 25% → leads to the idea of a self-check-in kiosk
- Self-service technologies (SSTs) are now used across many areas of business
- Typically come from: the funding sponsor, the acquiring customer, the manager of the actual users, marketing, or a product visionary

3.2 User Requirements

- Describe the **goals or tasks** users must be able to perform with the product to get value
- Include product attributes important to user satisfaction
- Represented through **use cases** and **user stories**
- Provided by user representatives
- Describe **what the user will be able to do** with the system
- **Example:**
  - Use case: *"Check in for a flight"* (via website or airport kiosk)
  - User story: *"As a passenger, I want to check in for a flight so I can board my airplane."*

3.3 Functional Requirements

- Specify the **behaviors** the product exhibits under specific conditions
- Describe what developers must implement so users can complete their tasks (thereby satisfying the business requirements)
- Often written as **"shall" statements**, e.g.:
  - *"The Passenger shall be able to print boarding passes for all flight segments for which he has checked in."*
  - *"If the Passenger's profile does not indicate a seating preference, the reservation system shall assign a seat."*
- A **constraint** restricts the choices available to the developer, e.g.:
  - *"The system shall be developed using open source tools and shall run on Linux OS."*

3.4 Working with the Three Levels

The three levels connect together: business requirements drive user requirements, which drive functional requirements. *(The original slide includes a diagram illustrating this flow.)*

---

4. Other Types of Requirements

4.1 System Requirements

- Describe requirements for a product made up of **multiple components or subsystems**
- A system can be all software, or a mix of software and hardware (e.g., biometric devices)
- People and processes are part of a system too — some functions may be allocated to humans
- **Example:** A supermarket cashier's workstation — bar code scanner integrated with a scale, a hand-held scanner, keyboard, display, and cash drawer

- System-level requirements are broken down by the business analyst into specific functionality allocated across subsystems, requiring a clear understanding of the interfaces between them

4.2 Non-Functional Requirements

- Also called **quality attributes** — describe a service or performance characteristic of the product (the "-ity, -ilities")
- Cover dimensions like performance, safety, availability, and portability (conflicts between these are fairly common)
- Some describe **external interfaces** between the system and the outside world — usefulness, flexibility, reliability, interoperability
- Include connections to other software systems, hardware components, users, and communication interfaces
- **Design and implementation constraints** restrict the developer's options during construction (e.g., security requirements)

4.3 Business Rules

- Include corporate policies, government regulations, industry standards, and computational algorithms
- **Not** software requirements themselves — they exist beyond the boundaries of any specific application
- Often require the system to implement functionality for compliance (e.g., an online purchase limit)
- Can act as a **constraint** on developer choices (e.g., no red color on a webpage — an "inverse requirement")
- Can be the origin of specific quality attributes later implemented as functionality (e.g., a corporate security policy leading to a firewall)
- You can often trace a functional requirement's origin back to a specific business rule

4.4 Feature

- One or more **logically related system capabilities** that provide value to a user, described by a set of functional requirements
- A feature can span **multiple user requirements**, each of which implies certain functional requirements needed to complete that task

---

5. Product vs. Project Requirements

5.1 Product Requirements

- Requirements describing properties of the **software system to be built**
- An **SRS** (Software Requirements Specification) houses product requirements only — it should **not** include design/implementation details (beyond known constraints), project plans, or test plans

- These other items should be separated out so requirements work can focus on *what* the team intends to build

5.2 Project Requirements

Projects have other expectations and deliverables that aren't part of the software itself, but are necessary for the project's success:

- Physical resources the team needs — workstations, special hardware, testing labs/tools, team rooms, videoconferencing equipment
- Staff training — user documentation, tutorials, reference manuals, release notes
- Infrastructure changes needed in the operating environment
- Requirements/procedures for releasing, installing, configuring, and testing the installation
- Beta testing, manufacturing, packaging, marketing, and distribution requirements
- Customer service-level agreements
- Requirements for legal protection (patents, trademarks, copyrights) of related intellectual property

---

6. Requirements Development Process

Requirements development has **four stages**: Elicitation → Analysis → Specification → Validation.

6.1 Elicitation

- Identifying the product's expected user classes and other stakeholders
- Understanding user tasks and goals, and the business objectives they align with
- Learning about the environment where the new product will be used
- Working with representatives of each user class to understand their functionality needs and quality expectations

6.2 Analysis

- Modeling the application environment
- Analyzing information from users to separate task goals into functional requirements, quality expectations, business rules, suggested solutions, etc.
- Decomposing high-level requirements into an appropriate level of detail
- Allocating requirements to software components defined in the system architecture
- Negotiating requirement priority and implementation order

6.3 Specification

- The business analyst documents requirements in a **Software Requirements Specification (SRS)**
- Transcribes collected user needs into written requirements and diagrams suitable for review by their intended audience

- The SRS describes the expected behavior of the system as fully as necessary
- Used across development, testing, QA, project management, and related functions
- Involves adopting requirement document templates, identifying requirement origins, and uniquely labeling each requirement

6.4 Validation

- Reviewing the documented requirements (SRS) to fix problems before development accepts them — checking they are **feasible, consistent, and complete**
  - *Consistency* = no contradictory requirements
  - *Completeness* = no missing services or constraints (also discard unnecessary requirements)
- Developing acceptance tests/criteria to confirm the product will meet customer needs and business objectives
- Simulating the requirements (e.g., wireframing) to catch errors early

(The deck includes several diagrams of this process framework, including a version time-boxed to one month.)

---

7. Requirements Management

- Establish a requirements **change control process**
- Perform **impact analysis** on requirement changes
- Track the status of each requirement and any related issues
- Maintain a **history** of requirement changes
- Maintain a **requirements traceability matrix**:
  - Defining relationships and dependencies between requirements
  - Tracing individual requirements to their corresponding designs, source code, and tests

---

8. Importance of Requirements

8.1 Role of Requirements

A well-known idea in the field: deciding **precisely what to build** is the hardest part of building a software system — harder than any other conceptual work, including all interfaces to people, machines, and other software systems. Getting this wrong cripples the resulting system more than any other mistake, and it's the hardest thing to fix later.

8.2 Reasons Behind Bad Requirements

- Insufficient user involvement
- Inaccurate planning
- **Creeping** user requirements (requirements that gradually increase or change)
- Ambiguous requirements (e.g., "respond to a request as soon as possible")
- **Gold plating** (adding extra functionality beyond the specification)

- Overlooked stakeholders

8.3 Benefits of a High-Quality Requirements Process

- Fewer defects in requirements and in the delivered product
- Reduced development rework
- Faster development and delivery
- Fewer unnecessary and unused features
- Lower enhancement costs
- Fewer miscommunications
- Reduced scope creep
- Reduced project chaos
- Higher customer and team member satisfaction
- Products that actually do what they're supposed to do

---

9. References

- Wiegers, K., & Beatty, J. (2013). *Software Requirements*. Pearson Education.
- CCSU CS530 Course Notes: Requirements